

COMP 3311

DATABASE MANAGEMENT SYSTEMS

LECTURE 12 EXERCISES

NOSQL

DATABASE MANAGEMENT SYSTEMS

BOATRESERVATIONS RELATIONAL DATABASE

Sailor(sailorId, sName, hkid, rating, age)

Reserves(sailorId, boatId, rDate)

Boat(boatId, bName, color, bType)

Attribute names in italics are foreign key attributes.

Sailor

<u>sailorId</u>	sName	hkid	rating	age
10	Emily	P936219(3)	null	16
15	Zoey	K091876(5)	null	17
22	Dustin	A298456(6)	7	45
29	Brutus	D746239(0)	1	33
31	Lucy	P930281(5)	10	55
32	Andy	M028391(4)	8	25
58	Rachael	A819204(3)	10	17
64	Horatio	F710943(2)	7	35
71	Zorba	P910562(4)	1	16
74	Horatio	T810463(1)	9	35
85	Amy	B719037(8)	3	25
95	Bob	E012735(5)	3	63
99	Chris	F710943(2)	10	30

13 tuples

Reserves

<u>sailorId</u>	<u>boatId</u>	<u>rDate</u>
22	103	08-SEP-24
22	104	07-OCT-24
22	101	10-OCT-24
22	102	10-NOV-24
31	102	10-OCT-24
31	103	06-NOV-24
31	102	12-NOV-24
64	101	05-SEP-24
64	102	08-SEP-24
74	103	08-SEP-24
99	102	08-AUG-24

11 tuples

Boat

<u>boatId</u>	bName	color	bType
101	Interlake	blue	dinghy
102	Interlake	red	sloop
103	Clipper	green	dinghy
104	Marine	red	catamaran
105	Serenity	cyan	null

5 tuples

EXERCISE 1

Construct a JSON object for each tuple in the Sailor relation with fields all the Sailor relation attributes; a field whose value is null should be absent. Order the query result objects by sailor id ascending.

```
select json_object(  
    'sailorId' value sailorId,  
    'sName' value sName,  
    'hkid' value hkid,  
    'rating' value rating,  
    'age' value age  
    absent on null)  
from Sailor  
order by sailorId;
```

Recall

The default behaviour for `json_object` is null on null.

The rating field will be absent for sailors whose rating is null.

EXERCISE 1 (CONTD)

Query result

{"sailorId":10,"sName":"Emily","hkid":"P936219(3)","age":16}
{"sailorId":15,"sName":"Zoey","hkid":"K091876(5)","age":17}
{"sailorId":22,"sName":"Dustin","hkid":"A298456(6)","rating":7,"age":45}
{"sailorId":29,"sName":"Brutus","hkid":"D746239(0)","rating":1,"age":33}
{"sailorId":31,"sName":"Lucy","hkid":"P930281(5)","rating":10,"age":55}
{"sailorId":32,"sName":"Andy","hkid":"M028391(4)","rating":8,"age":25}
{"sailorId":58,"sName":"Rachael","hkid":"A819204(3)","rating":10,"age":17}
{"sailorId":64,"sName":"Horatio","hkid":"F710943(2)","rating":7,"age":35}
{"sailorId":71,"sName":"Zorba","hkid":"P910562(4)","rating":1,"age":16}
{"sailorId":74,"sName":"Horatio","hkid":"T810463(1)","rating":9,"age":35}
{"sailorId":85,"sName":"Amy","hkid":"B719037(8)","rating":3,"age":25}
{"sailorId":95,"sName":"Bob","hkid":"E012735(5)","rating":3,"age":63}
{"sailorId":99,"sName":"Chris","hkid":"H814243(7)","rating":10,"age":30}

EXERCISE 2

Construct a JSON array for each tuple in the Sailor relation with elements all the values of the Sailor tuple ordered by rating descending. An array element whose value is null should be included.

```
select json_array(  
    sailorId,  
    sName,  
    hkid,  
    rating,  
    age  
    null on null)  
from Sailor  
order by rating desc;
```

Recall

The default behaviour for `json_array` is **absent on null**.

The query result will contain one array for each Sailor tuple.
The rating array element will be null for sailors whose rating is null.

EXERCISE 2 (CONTD)

Query result

[10,"Emily","P936219(3)",null,16]
[15,"Zoey","K091876(5)",null,17]
[58,"Rachael","A819204(3)",10,17]
[99,"Chris","H814243(7)",10,30]
[31,"Lucy","P930281(5)",10,55]
[74,"Horatio","T810463(1)",9,35]
[32,"Andy","M028391(4)",8,25]
[64,"Horatio","F710943(2)",7,35]
[22,"Dustin","A298456(6)",7,45]
[95,"Bob","E012735(5)",3,63]
[85,"Amy","B719037(8)",3,25]
[71,"Zorba","P910562(4)",1,16]
[29,"Brutus","D746239(0)",1,33]

EXERCISE 3

Construct a JSON object for each tuple in the Sailor relation with fields sailor id, name and reservations which is an object containing, as its fields, the field boat id and its value for each boat that the sailor has reserved; if a sailor has no reservations, then the "reservations" field should be absent.

Is this a correct solution?

Partially!

Why?

```
select json_object(  
  'sailorId'    value sailorId,  
  'sName'      value sName,  
  'reservations' value json_objectagg('boatId' value boatId absent on null)  
                absent on null)  
from Sailor natural left outer join Reserves  
group by sailorId, sName;
```

The query result contains all reservations made by a sailor, but the `reservations` field will be present for all sailors with an empty `reservations` object for those sailors who have no reservations.

👉 The second `absent on null` specification does not produce the desired result. Why?

How to get no reservations field for sailors with no reservations?

EXERCISE 3 (CONT'D)

Construct a JSON object for each tuple in the Sailor relation with fields sailor id, name and reservations which is an object containing, as its fields, the field boat id and its value for each boat that the sailor has reserved; if a sailor has no reservations, then the "reservations" field should be absent.

```
select json_object(  
  'sailorId'    value sailorId,  
  'sName'      value sName,  
  'reservations' value (select json_objectagg('boatId' value boatId)  
                        from Reserves R  
                        where R.sailorId=S.sailorId)  
  absent on null)  
from Sailor S;
```

The reservations field will be absent for sailors who have no reservations.

Recall

The default behaviour for `json_objectagg` is null on null.

EXERCISE 3 (CONTD)

Query result

{"sailorId":10,"sName":"Emily"}
{"sailorId":15,"sName":"Zoey"}
{"sailorId":22,"sName":"Dustin","reservations":{"boatId":101,"boatId":102,"boatId":103,"boatId":104}}
{"sailorId":29,"sName":"Brutus"}
{"sailorId":31,"sName":"Lucy","reservations":{"boatId":102,"boatId":102,"boatId":103}}
{"sailorId":32,"sName":"Andy"}
{"sailorId":58,"sName":"Rachael"}
{"sailorId":64,"sName":"Horatio","reservations":{"boatId":101,"boatId":102}}
{"sailorId":71,"sName":"Zorba"}
{"sailorId":74,"sName":"Horatio","reservations":{"boatId":103}}
{"sailorId":85,"sName":"Amy"}
{"sailorId":95,"sName":"Bob"}
{"sailorId":99,"sName":"Chris","reservations":{"boatId":102}}

EXERCISE 4

Construct a JSON object for each tuple in the Sailor relation with fields sailor id, name and "reserved boats" which is an array field whose elements are the boat id values, in ascending order, of the boats that the sailor has reserved. If a sailor has not reserved any boat, then the "reserved boats" field should be absent. Order the query result objects by sailor id ascending.

Is this a
correct
solution?

Partially!
Why?

```
select json_object(  
  'sailorId'      value sailorId,  
  'sName'        value sName,  
  'reserved boats' value json_arrayagg(boatId order by boatId absent on null)  
  absent on null)  
from Sailor natural left outer join Reserves  
group by sailorId, sName  
order by sailorId;
```

How to
get an
absent
reserved
boats
field?

The query result contains all reservations made by a sailor, but the reserved boats field will be present for all sailors with an empty array for those sailors who have no reservations.

👉 The second **absent on null** specification does not produce the desired result. Why?

Recall

The default behaviour for `json_arrayagg` is absent on null.

EXERCISE 4 (CONT'D)

Construct a JSON object for each tuple in the Sailor relation with fields sailor id, name and "reserved boats" which is an array field whose elements are the boat id values, in ascending order, of the boats that the sailor has reserved. If a sailor has not reserved any boat, then the "reserved boats" field should be absent. Order the query result objects by sailor id ascending.

```
select json_object(  
  'sailorId'      value sailorId,  
  'sName'        value sName,  
  'reserved boats' value (select json_arrayagg(boatId order by boatId)  
                           from Reserves R  
                           where R.sailorId=S.sailorId)  
  absent on null)  
from Sailor S  
order by sailorId;
```

The reserved boats field will be absent for sailors who have no reservations.

EXERCISE 4 (CONT'D)

Query result

{"sailorId":10,"sName":"Emily"}
{"sailorId":15,"sName":"Zoey"}
{"sailorId":22,"sName":"Dustin","reserved boats":[101,102,103,104]}
{"sailorId":29,"sName":"Brutus"}
{"sailorId":31,"sName":"Lucy","reserved boats":[102,102,103]}
{"sailorId":32,"sName":"Andy"}
{"sailorId":58,"sName":"Rachael"}
{"sailorId":64,"sName":"Horatio","reserved boats":[101,102]}
{"sailorId":71,"sName":"Zorba"}
{"sailorId":74,"sName":"Horatio","reserved boats":[103]}
{"sailorId":85,"sName":"Amy"}
{"sailorId":95,"sName":"Bob"}
{"sailorId":99,"sName":"Chris","reserved boats":[102]}

EXERCISE 5

Construct a JSON object for each sailor who has reserved a boat with fields sailor id, name and reservations which is an array field whose values are objects, one for each of the sailor's reservations, with fields boat id, boat name and boat type. Order the query result objects by sailor id ascending.

```
select json_object(  
    'sailorId'    value sailorId,  
    'sName'      value sName,  
    'reservations' value json_arrayagg(json_object('boatId' value boatId,  
                                                    'bName' value bName,  
                                                    'bType' value bType)))  
from Sailor natural join Reserves natural join Boat  
group by sailorId, sName  
order by sailorId;
```

EXERCISE 5 (CONTD)

Query result

<pre>{"sailorId":22,"sName":"Dustin","reservations":[{"boatId":101,"bName":"Interlake","bType":"dinghy"}, {"boatId":104,"bName":"Marine","bType":"catamaran"}, {"boatId":103,"bName":"Clipper","bType":"dinghy"}, {"boatId":102,"bName":"Interlake","bType":"sloop"}]}</pre>
<pre>{"sailorId":31,"sName":"Lucy","reservations":[{"boatId":102,"bName":"Interlake","bType":"sloop"}, {"boatId":103,"bName":"Clipper","bType":"dinghy"}, {"boatId":102,"bName":"Interlake","bType":"sloop"}]}</pre>
<pre>{"sailorId":64,"sName":"Horatio","reservations":[{"boatId":101,"bName":"Interlake","bType":"dinghy"}, {"boatId":102,"bName":"Interlake","bType":"sloop"}]}</pre>
<pre>{"sailorId":74,"sName":"Horatio","reservations":[{"boatId":103,"bName":"Clipper","bType":"dinghy"}]}</pre>
<pre>{"sailorId":99,"sName":"Chris","reservations":[{"boatId":102,"bName":"Interlake","bType":"sloop"}]}</pre>

EXERCISE 6

Construct a JSON object for each boat with fields all the attributes of Boat and reservations which is an array field whose values are objects, one for each of the boat's reservations, with fields sailor id, sailor name and rating. If a boat has never been reserved, then the "reservations" field should be absent.

Is this a
correct
solution?

No!

Why?

```
select json_object(  
  'boatId'      value boatId,  
  'bName'       value bName,  
  'color'       value color,  
  'bType'       value bType,  
  'reservations' value json_arrayagg(json_object('sailorId' value sailorId,  
                                                  'sName' value sName,  
                                                  'rating' value rating,  
                                                  absent on null))  
  absent on null)  
from Boat natural left outer join Reserves natural left outer join Sailor  
group by boatId, bName, color, bType;
```

The reservations array field will be present with an empty object element for those boats that have no reservations.

EXERCISE 6 (CONT'D)

Construct a JSON object for each boat with fields all the attributes of Boat and reservations which is an array field whose values are objects, one for each of the boat's reservations, with fields sailor id, sailor name and rating. If a boat has never been reserved, then the "reservations" field should be absent.

```
select json_object(  
  'boatId'      value boatId,  
  'bName'       value bName,  
  'color'       value color,  
  'bType'       value bType,  
  'reservations' value (select json_arrayagg(json_object(  
                        'sailorId' value S.sailorId,  
                        'sName'   value sName,  
                        'rating'  value rating))  
                        from Sailor S, Reserves R  
                        where S.sailorId=R.sailorId and R.boatId=B.boatId)  
  absent on null)  
from Boat B;
```

The reservations array field will be absent for boats with no reservations.

EXERCISE 6 (CONT'D)

Query result

<pre>{"boatId":101,"bName":"Interlake","color":"blue","bType":"dinghy","reservations":[{"sailorId":22,"sName":"Dustin","rating":7}, {"sailorId":64,"sName":"Horatio","rating":7}]}</pre>
<pre>{"boatId":102,"bName":"Interlake","color":"red","bType":"sloop","reservations":[{"sailorId":22,"sName":"Dustin","rating":7}, {"sailorId":31,"sName":"Lucy","rating":10}, {"sailorId":31,"sName":"Lucy","rating":10}, {"sailorId":64,"sName":"Horatio","rating":7}, {"sailorId":99,"sName":"Chris","rating":10}]}</pre>
<pre>{"boatId":103,"bName":"Clipper","color":"green","bType":"dinghy","reservations":[{"sailorId":22,"sName":"Dustin","rating":7}, {"sailorId":31,"sName":"Lucy","rating":10}, {"sailorId":74,"sName":"Horatio","rating":9}]}</pre>
<pre>{"boatId":104,"bName":"Marine","color":"red","bType":"catamaran","reservations":[{"sailorId":22,"sName":"Dustin","rating":7}]}</pre>
<pre>{"boatId":105,"bName":"Serenity","color":"cyan"}</pre>

EXERCISE 7

Construct a JSON object for each sailor with fields all the attributes of Sailor and reservations which is an array field whose values are objects, one for each of the sailor's reservations, with fields boat id and reservation date. If a sailor has not reserved any boat, then the "reservations" field should be absent. The reservation date format should be the same as in the Reserves relation, i.e., DD-MON-YY.

```
select json_object(  
  'sailorId' value sailorId,  
  'sName' value sName,  
  'hkid' value hkid,  
  'rating' value rating,  
  'age' value age,  
  'reservations' value (select json_arrayagg(json_object('boatId' value boatId,  
                                                         'rDate' value to_char(rDate,'DD-MON-YY')))  
                        from Reserves R  
                        where R.sailorId=S.sailorId)  
  absent on null)  
from Sailor S;
```



EXERCISE 7 (CONTD)

Query result

{"sailorId":10,"sName":"Emily","hkid":"P936219(3)","age":16}
{"sailorId":15,"sName":"Zoey","hkid":"K091876(5)","age":17}
{"sailorId":22,"sName":"Dustin","hkid":"A298456(6)","rating":7,"age":45,"reservations":[{"boatId":101,"rDate":"10-OCT-25"}, {"boatId":102,"rDate":"10-NOV-25"}, {"boatId":103,"rDate":"08-SEP-25"}, {"boatId":104,"rDate":"10-OCT-25"}]}
{"sailorId":29,"sName":"Brutus","hkid":"D746239(0)","rating":1,"age":33}
{"sailorId":22,"sName":"Dustin","hkid":"A298456(6)","rating":7,"age":45,"reservations":[{"boatId":101,"rDate":"10-OCT-25"}, {"boatId":102,"rDate":"10-NOV-25"}, {"boatId":103,"rDate":"08-SEP-25"}, {"boatId":104,"rDate":"10-OCT-25"}]}
{"sailorId":32,"sName":"Andy","hkid":"M028391(4)","rating":8,"age":25}
{"sailorId":58,"sName":"Rachael","hkid":"A819204(3)","rating":10,"age":17}
{"sailorId":64,"sName":"Horatio","hkid":"F710943(2)","rating":7,"age":35,"reservations":[{"boatId":101,"rDate":"05-SEP-25"}, {"boatId":102,"rDate":"08-SEP-25"}]}
{"sailorId":71,"sName":"Zorba","hkid":"P910562(4)","rating":1,"age":16}
{"sailorId":74,"sName":"Horatio","hkid":"T810463(1)","rating":9,"age":35,"reservations":[{"boatId":103,"rDate":"08-SEP-25"}]}
{"sailorId":85,"sName":"Amy","hkid":"B719037(8)","rating":3,"age":25}
{"sailorId":95,"sName":"Bob","hkid":"E012735(5)","rating":3,"age":63}
{"sailorId":99,"sName":"Chris","hkid":"H814243(7)","rating":10,"age":30,"reservations":[{"boatId":102,"rDate":"08-AUG-25"}]}